

MHCP Mentors

What Makes Your AI Tick?

A Deep Dive on Understanding AI Agents

April 2026

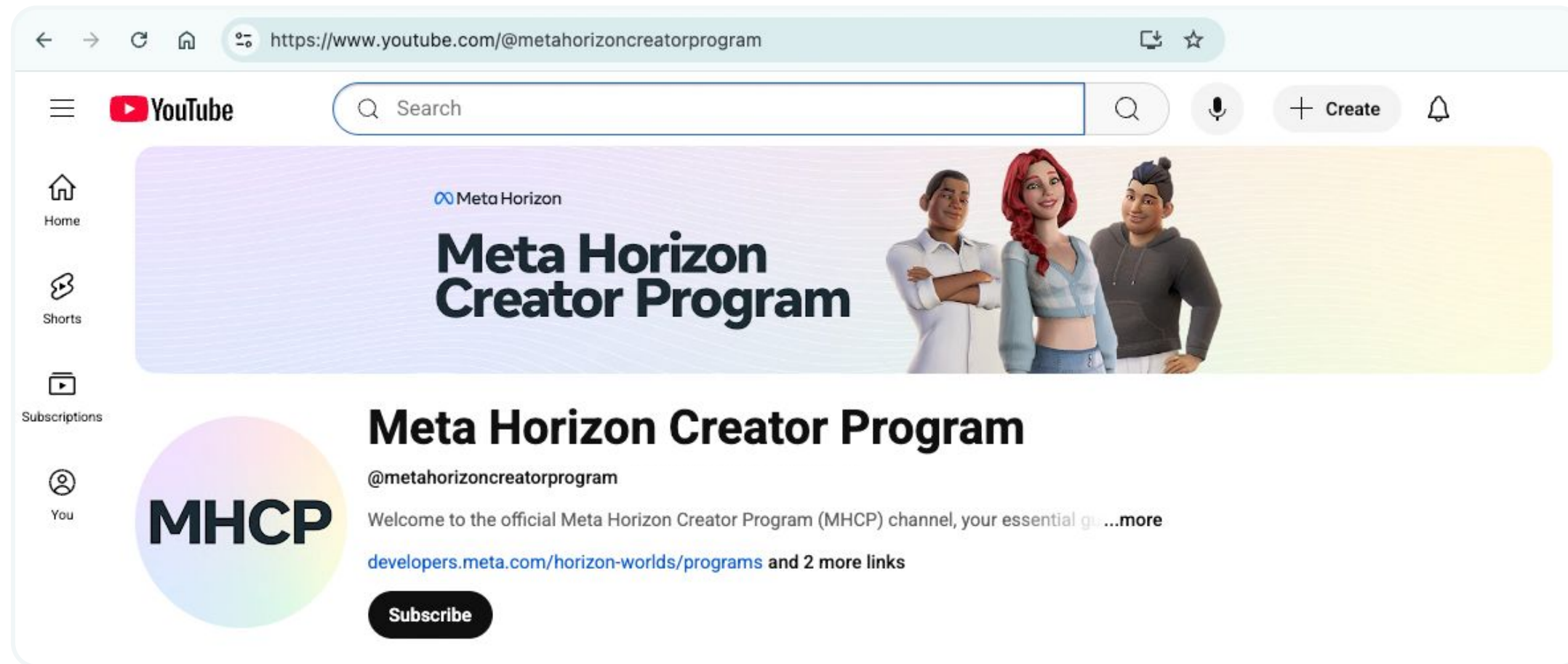
Shards632
AI Psychologist



Callouts

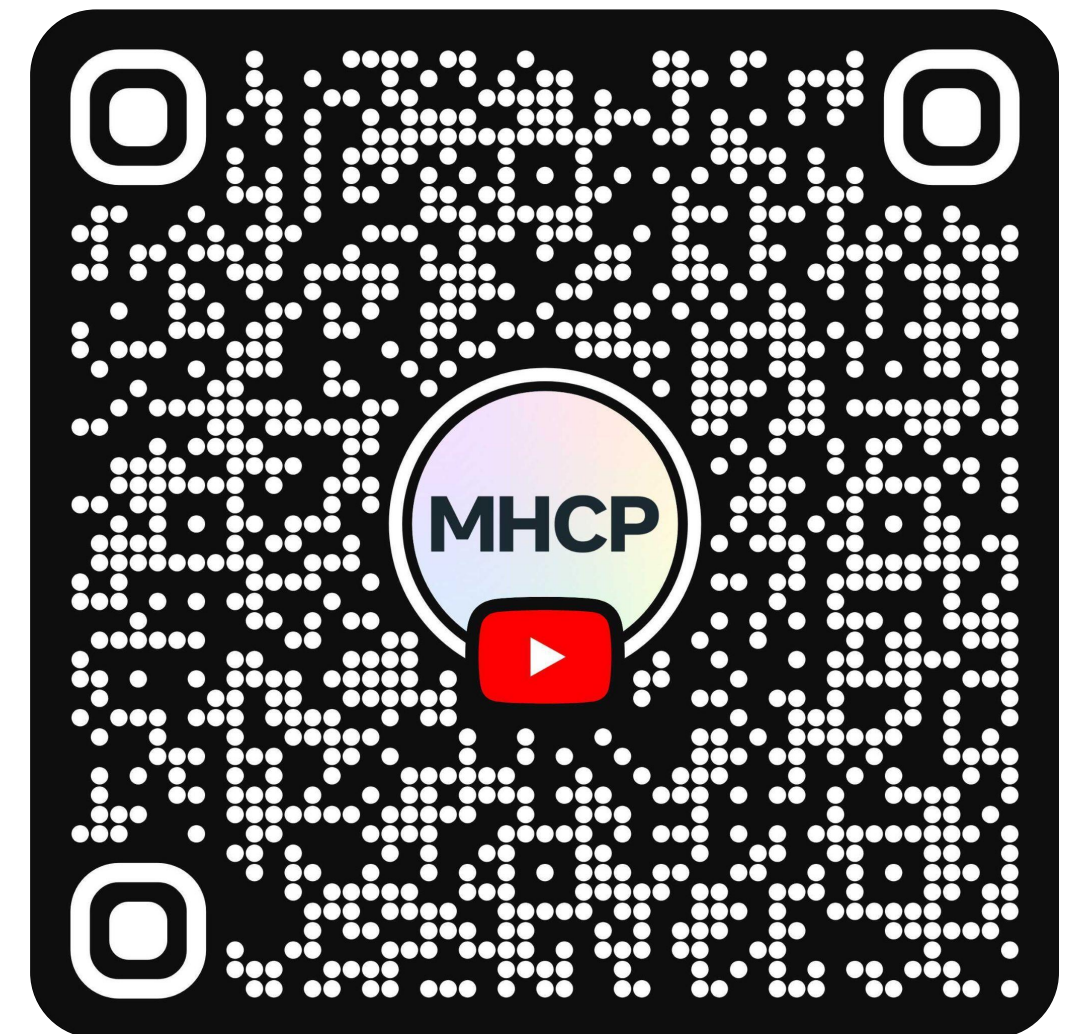
Worlds Playtest: VR-to-Mobile
Thursday April 30th, 11am PDT, 2pm EDT
Gaus Roth

May Events Coming Soon!



<https://www.youtube.com/@metahorizoncreatorprogram>

 **YouTube**
Subscribe now



Agenda

- LLM Architecture
- Context (Window) is King
- Subagents
- Plugins
- Q&A

Skills Matrix alignment

Design	Project Management 	Scripting 	Art	Operations
Mechanics The main interactions, rules, procedures that a player uses / interacts with (that are fun / delightful / useful) via items, interactions, UI, etc.	Product Management  The frameworks for turning ideas into practical, executable projects with clear goals, timelines, and deliverables	Programming  The skills needed to write and manage (3D content) codebases.	Assets The design and creation of static meshes for use as world environments or items / props / wearables / characters (including meshes and textures).	Marketing The tools for acquiring and managing an audience.
Core Loop The creation of meaning / accomplishment / feedback when repeatedly using the (main) mechanics.	Content Development  The processes to ensure an idea is clear and “proven” before moving to the long stage of polished script and art creation.	Meta Horizon Systems The use of Meta Horizon’s specific technical features and capabilities.	Dynamics The creation and application of elements that bring life to static art (such as animations, visual effects, lighting, sound effects, and music).	Business Management The administration of team / financial management, including payments, legal affairs, and related human resources functions (including managing partnerships and talent).
Progression The frameworks that allow players to accrue long-term value and retain in a world (achievements, identify, items, etc.).				

Mentor Workshop important info:



This session is recorded—the session will be posted on YouTube later.



Raise your hand—use the Zoom “Raise your hand” function to ask a question (or put it in the chat).



Mute yourself—keep muted unless you are asking a question.



Keep the conversation going! Post your work or questions in the forum.



Questions at designated times—save most questions for the Q&A portions.



Be active in the chat—share relevant insights and help answer others’ questions when possible.



Use the chat for technical issues—report technical problems in the chat, not verbally.



Fill out the feedback survey—your input helps us improve future workshops.



Tag me in the appropriate channels for feedback. Please don’t DM me. (I won’t answer.)

LLM Architecture

Overview

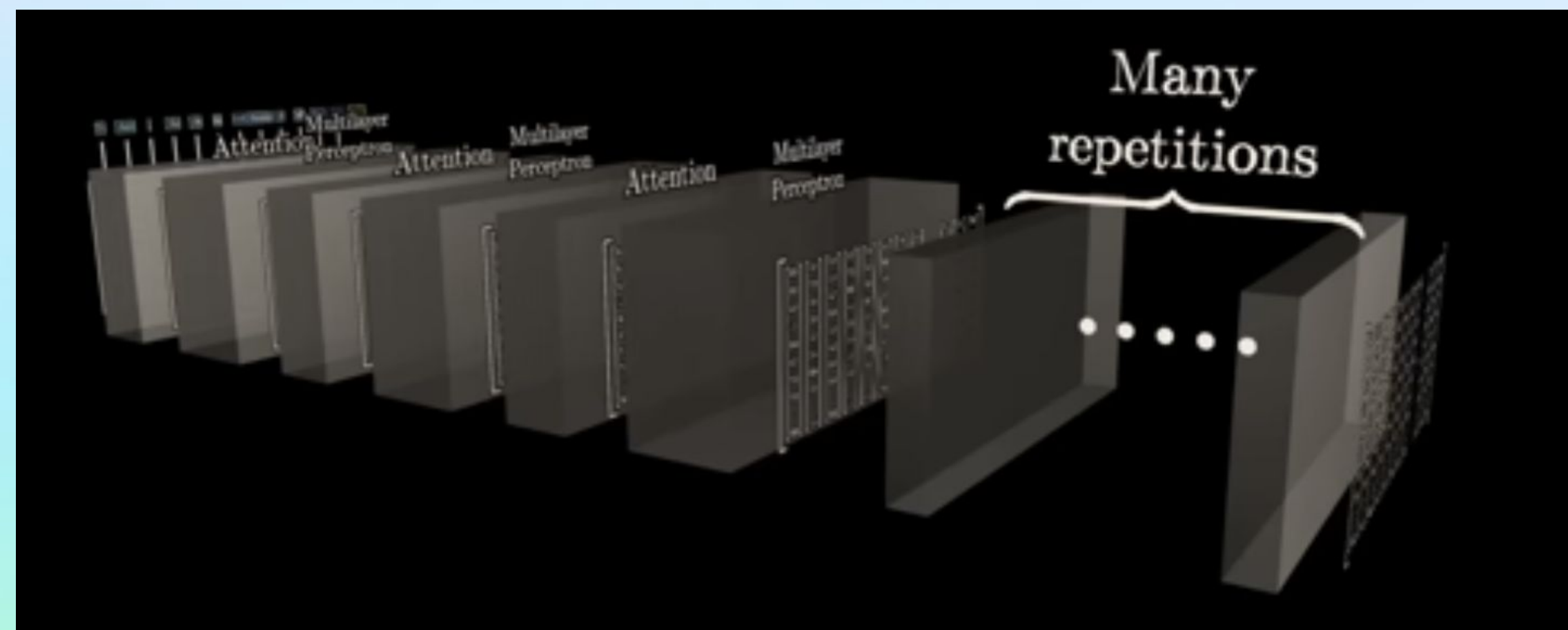
Embedding

Attention Heads

Perceptrons

Predict Next Token

An LLM is a very large and deep sequence of layers that translates your text input into a long sequence of high dimensional space vectors, manipulates those vectors in various interdependent ways, all to figure out what the most likely next token is after the entire sequence is processed.



LLM Architecture

Overview

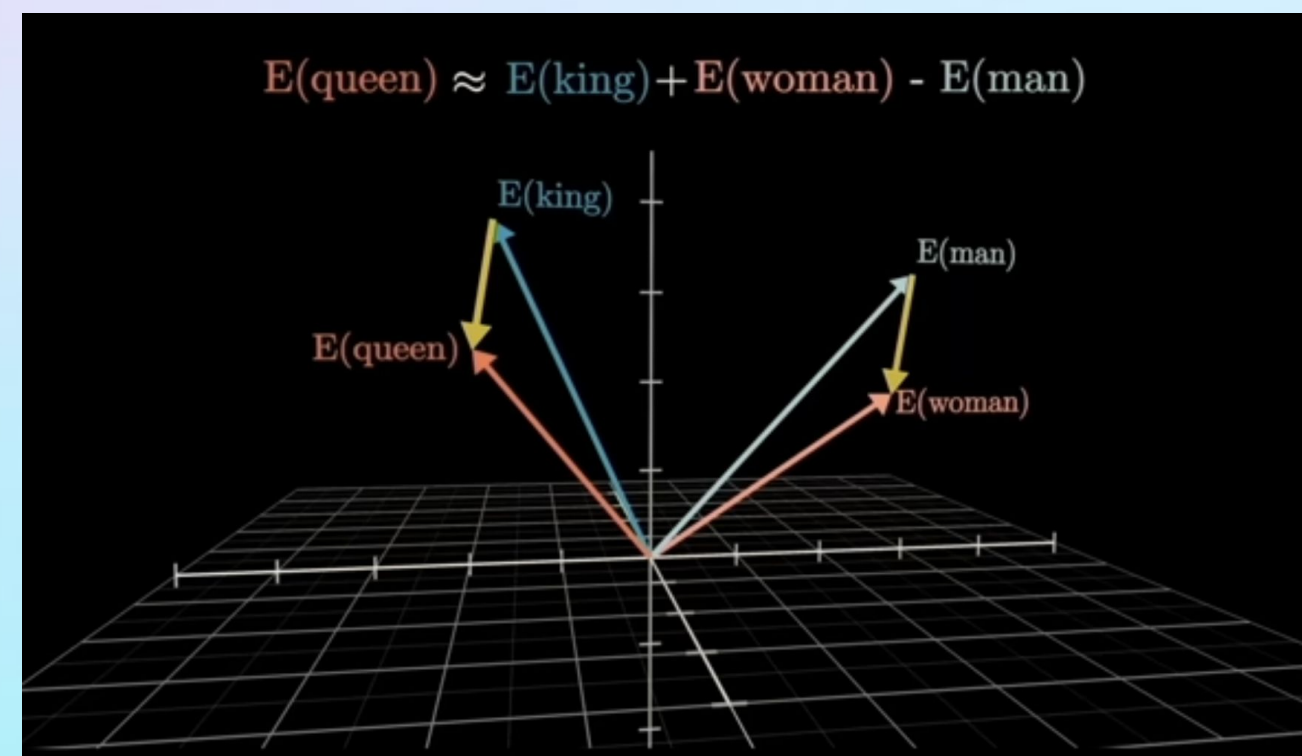
Embedding

Attention Heads

Perceptrons

Predict Next Token

The words of your input are tokenized into pieces. Typically a modern model can interpret about 50,000+ unique tokens (roughly words, or word parts), embedded into a 5,000+ dimensional space.



These token sequences constructed from your input are then fed in to the model, in order.

LLM Architecture

Overview

Embedding

Attention Heads

Perceptrons

Predict Next Token

The critical advancement of modern LLM technology is 'attention'. This is a pass in the LLM architecture where the model computes how correlated tokens later in the sequence are to embedding vectors that came before them.

Attention heads are typically 100+ dimensions, much smaller than the embedding space.

There are many different ways attention heads work in an LLM to extract different kinds of correlations. The way the attention heads work is "learned" by the model during training on gigantic amounts of data.

LLM Architecture

Overview

Embedding

Attention Heads

Perceptrons

Predict Next Token

This layer of the LLM independently converts **each** attention boosted token in the stream to a stream of new embeddings representing a different "concept" in the high dimensional embedding space.

Perceptrons contain multiple hidden layers that operate at higher dimensions than the embedding (10000+ dimensions)

This is where the "knowledge" of the LLM is stored. There are many different kinds of perceptron networks in an LLM model. The way they work on the embeddings is, again, "learned" by the model during the intensive training phase on massive amounts of data.

LLM Architecture

Overview

Embedding

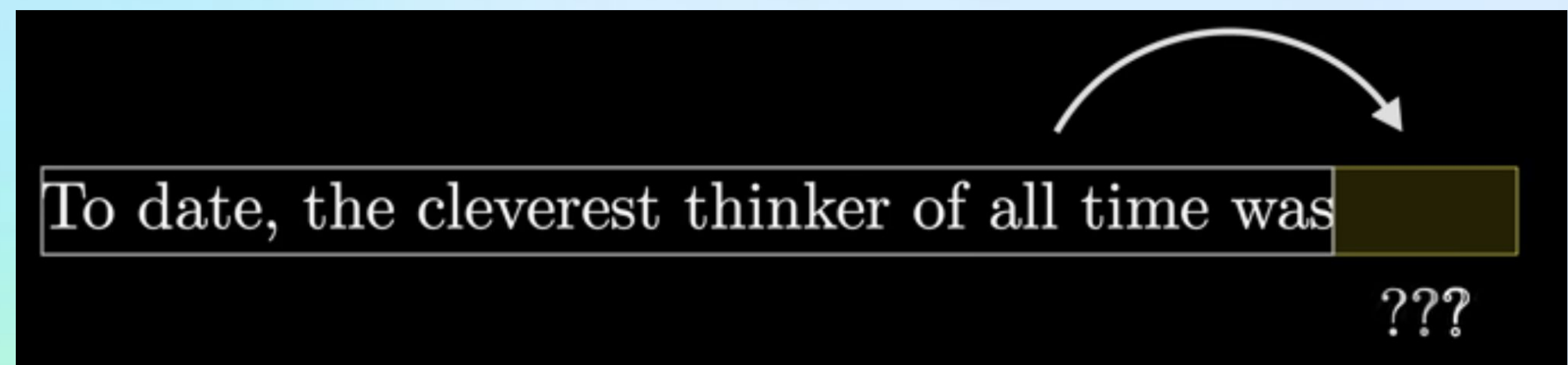
Attention Heads

Perceptrons

Predict Next Token

Each layer of the LLM can have 100+ attention heads, and there can be 100+ stacked layers of attention head / perceptron units.

The final output of the LLM is to predict, given a sequence of tokens seen up to this point, what is the most likely token to come next. This is performed on the entire input sequence, but only the output after the last token is used to predict the next word (after de-embedding).



Context (Window) is King

Limited size

Lost context

Management

The translation of a prompt into a sequence of embedding vectors is a key limitation to how well an LLM model works.

Given the way attention heads work, tokens that come early in that sequence have a big influence on how they influence later tokens attention.

Given the way the LLM works overall in predicting the last token in a sequence, later tokens in the sequence have a very large impact on the 'next token' prediction.

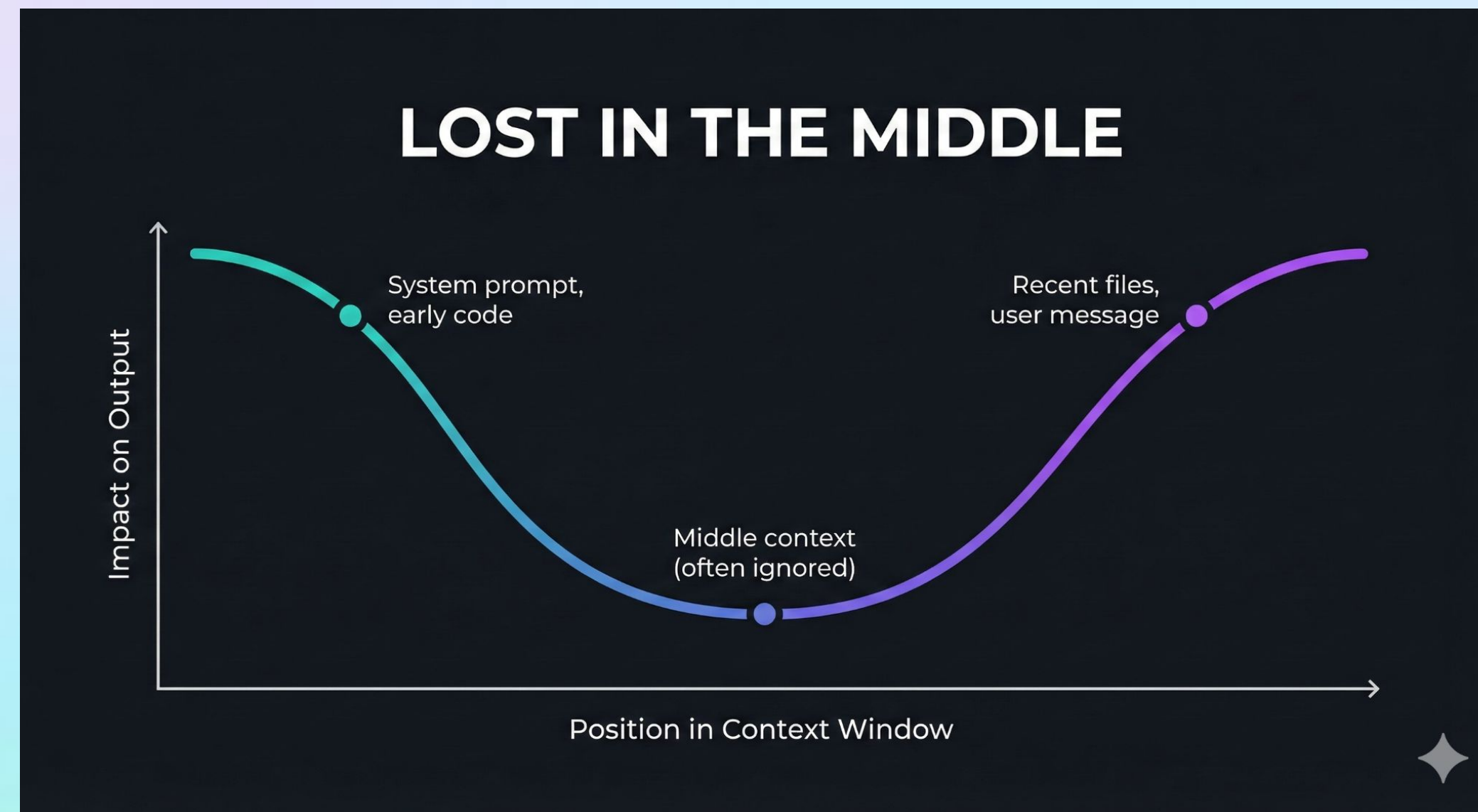
Context (Window) is King

Limited size

Lost Context

Management

Tokens in the middle can get 'lost'. If you exceed the window, early tokens can get dropped (forgetting).



Context (Window) is King

Limited size

Lost Context

Management

You must aggressively manage your token window:

- Avoid adding tokens that are not needed for your task at hand.
 - Keep Claude.md short!
- Avoid long conversations
- Monitor token window usage
 - Typically models degrade after 50% or so
 - Use `/context` frequently to see where you are
- Instruct LLM to produce *artifacts* (e.g. plan.md, status.md, etc.) to summarize conversations and track progress. Reference those artifacts in new chats.
- Work incrementally
 - frequently start new chats
 - Use `/clear` to restart the current chat

Subagents

What are they?

Configuration

Invocation

Agents are a way to focus a *different* instance of the LLM with it's own context window on a specific subtask.

Subagent can consume large parts of its ***own*** context window reading and processing files / data, and report back a concise result to the coordinating agent.

Coordinating agent can then digest that summary with minimal impact on its own context window.

Coordinating agent can also launch *multiple* subagents that run in parallel to work independently on non overlapping parts of your project.

Subagents

What are they?

Configuration

Invocation

- Subagents for Claude are described in `.claude/agents` directory
- Two sections
 - YAML frontmatter

```
---  
name: code-reviewer  
description: Reviews code for quality and best practices  
tools: Read, Glob, Grep  
model: sonnet  
---
```

- LLM instructions

```
You are a code reviewer. When invoked, analyze the code and provide  
specific, actionable feedback on quality, security, and best practices.
```

- Frontmatter is cached by main agent to learn about subagents for specific tasks without reading LLM instructions for subagent.
- You can use the LLM to help you write sub agents
- Lots of built in agents exist

Subagents

What are they?

Configuration

Invocation

- Agents can be invoked automatically by the coordinating agent when they recognize a request that matches the description of a subagent.
- Agents can also be launched explicitly by commands such as
 - "Run the following tasks in parallel in multiple subagents"

Plugins

What are they?

Skills

Hooks

MCP Servers

Plugins are useful for performing specific repetitive tasks.

They come in a few variants

- Skills
 - snippets of LLM instructions that you can perform as a sort of a 'macro'
- Hooks
 - snippets of LLM instructions that you want run at particular operation points or periodically
- MCP Servers
 - external programs that you want to surface to the LLM as though they were native capabilities

Plugins

What are they?

Skills

Hooks

MCP Servers

- Skills for Claude are described in `.claude/skills/<name>/SKILL.md`
- Make a skill when you find yourself writing the same instructions to your LLM over and over
- YAML frontmatter

```
---  
name: explain-code  
description: Explains code with visual diagrams and analogies.  
---
```

- 'name' will become the 'slash' command, e.g. `/explain-code`
- LLM instructions
 - A snippet of text describing the macro action to perform

Plugins

What are they?

Skills

Hooks

MCP Servers

- Hooks for Claude live in `.claude/settings.json`
- Can be configured to fire for various types of events (PreToolUse, SubagentStop, PreCompact etc.)
- Matcher pattern on event determines if tool applies
- If matcher matches, a handler is run
 - Handler can be:
 - shell script
 - mcp tool
 - LLM prompt
 - subagent launch
 - http request
- Useful to validate state when event happens for security or to generate side effects elsewhere
- Watch out for bloating of context window with too many hooks!

Plugins

What are they?

Skills

Hooks

MCP Servers

- MCP configuration for Claude is in `.claude/settings.json`
- Best managed using built in `claude /mpc` skill to add remove
- These are *EXTERNAL* servers that the LLM communicates with using HTTP connections. Typically, **NOT ON YOUR MACHINE!**
- MCPs can be *very* powerful, as they can make wide ranging side effects outside your machine, and tap deep knowledge sources outside of the training of your LLM
- Lots and lots of 3rd party MCP tools exist
 - e.g. GitHub MCP to manage reading/pushing/pulling code, creating pull requests, etc.
- Configuration info for too many MCPs can cause hidden bloat of context window. Manage just the set you need!

Resources

- Excellent discussion on how transformers work
 - <https://www.youtube.com/watch?v=KJtZARuO3JY>
 - 3Blue1Brown has lots of other excellent vids on a wide range of science-y topics!
- Claude subagent documentation
 - <https://code.claude.com/docs/en/sub-agents>
- Claude skills documentation
 - <https://code.claude.com/docs/en/skills>
- Claude hooks documentation
 - <https://code.claude.com/docs/en/hooks-guide>
- Popular MCP list
 - <https://code.claude.com/docs/en/mcp#popular-mcp-servers>

Q&A

Thank You!

Thank you for learning with me!

Please scan the QR
code to fill out the
feedback survey:



 **YouTube**
Subscribe

